05 — Feature extraction and latent representations

I fit PCA first as a linear baseline, then UMAP and ISOMAP on PCA-reduced V1 population activity. I use these embeddings as baseline manifolds before CEBRA and downstream decoding.

```
from pathlib import Path
import sys

ROOT = Path.cwd()
if ROOT.name == "notebooks":
    ROOT = ROOT.parent
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

from v1_manifold.config import load_config, get_paths, set_global_seed
from v1_manifold.visualization import set_publication_style, save_figure
from v1_manifold.utils import save_table

cfg = load_config(ROOT / "configs" / "default.yaml")
cfg["paths"]["root"] = str(ROOT)
paths = get_paths(cfg)
set_global_seed(cfg["project"]["random_seed"])
set_publication_style()

print(f"Project root: {ROOT}")
```

```
Project root: c:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

Load the processed neural tensor and real movie-frame features

I use the repeat-averaged population response matrix from notebook 03. Each row corresponds to one natural movie frame and each column corresponds to one retained V1 neuron.

I also load the real frame-feature table created in notebook 04. The analysis stops if fallback stimulus labels are present, because latent-feature interpretation should use real visual features extracted from the Allen natural movie pixels.

```
import numpy as np
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from v1_manifold.preprocessing import load_trial_tensor_h5
from v1_manifold.features import standardize_matrix, assert_real_stimulus_fe
from v1_manifold.manifold import fit_pca, fit_umap, fit_isomap, save_embeddi

h5_files = sorted(paths.interim_dir.glob("session_*_tensor.h5"))
if not h5_files:
    raise FileNotFoundError("No trial tensor exists yet. I need to run noteb

tensor_path = h5_files[0]
session_id = tensor_path.name.split("_")[1]

tensor, R = load_trial_tensor_h5(tensor_path)

real_feature_candidates = sorted(paths.processed_dir.glob(f"session_{session
generic_feature_candidates = sorted(paths.processed_dir.glob(f"session_{sess

if real_feature_candidates:
    feature_path = real_feature_candidates[0]
elif generic_feature_candidates:
    feature_path = generic_feature_candidates[0]
else:
    raise FileNotFoundError("Real frame features are missing. I need to run

features = pd.read_csv(feature_path)
features = features.sort_values("movie_frame").reset_index(drop=True)

assert_real_stimulus_features(features)

if R.shape[0] != len(features):
    raise ValueError(
        "The neural response matrix and feature table do not align: "
        f"R has {R.shape[0]} frames but features has {len(features)} rows."
    )

expected_frames = np.arange(R.shape[0])
if not np.array_equal(features["movie_frame"].to_numpy(), expected_frames):
    raise ValueError(
        "The feature table movie_frame column is not aligned to the neural m
        "I expected frames 0..n_frames-1 after sorting."
    )

Xz, scaler = standardize_matrix(R)
save_model(scaler, paths.models_dir / f"session_{session_id}_scaler.joblib")

print("Tensor shape:", tensor.shape)
print("Repeat-averaged matrix shape:", R.shape)
print("Feature table:", feature_path)
```

```
print("Feature columns:", len(features.columns))
display(features.head())
```

```
Tensor shape: (10, 163, 900)
Repeat-averaged matrix shape: (900, 163)
Feature table: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-n
Feature columns: 34
```

	movie_frame	population_mean	population_std	population_l2_norm	n_cells
0	0	0.459886	2.380123	30.949413	163
1	1	0.475428	2.378806	30.971182	163
2	2	0.436005	2.304510	29.943974	163
3	3	0.373745	2.170050	28.113255	163
4	4	0.420181	2.040942	26.603487	163

5 rows × 34 columns

Fit PCA, UMAP, and ISOMAP embeddings

I fit PCA as the linear baseline and compute the participation ratio as an effective dimensionality estimate. UMAP and ISOMAP are then fit on PCA scores so that nonlinear embedding is not dominated by raw high-dimensional noise.

```
pca_components = min(
    int(cfg["features"]["pca_components"]),
    Xz.shape[0],
    Xz.shape[1],
)

pca_scores, pca, pr = fit_pca(
    Xz,
    pca_components,
    cfg["project"]["random_seed"],
)

save_model(pca, paths.models_dir / f"session_{session_id}_pca.joblib")

embedding_dim = 3

pca_embedding = pca_scores[:, :embedding_dim]

umap_embedding = fit_umap(
    pca_scores,
```

```
    embedding_dim,
    cfg["features"]["umap_neighbors"],
    cfg["features"]["umap_min_dist"],
    cfg["project"]["random_seed"],
)

isomap_embedding = fit_isomap(
    pca_scores,
    embedding_dim,
    cfg["features"]["isomap_neighbors"],
)

emb_path = paths.processed_dir / f"session_{session_id}_embeddings.npz"

save_embedding_npz(
    emb_path,
    pca=pca_embedding,
    pca_full=pca_scores,
    umap=umap_embedding,
    isomap=isomap_embedding,
    frame=features["movie_frame"].to_numpy(),
    rms_contrast=features["rms_contrast"].to_numpy() if "rms_contrast" in fe
    spatial_frequency_centroid=features["spatial_frequency_centroid"].to_num
)

print(f"Participation ratio: {pr:.3f}")
print("Saved embeddings:", emb_path)
```

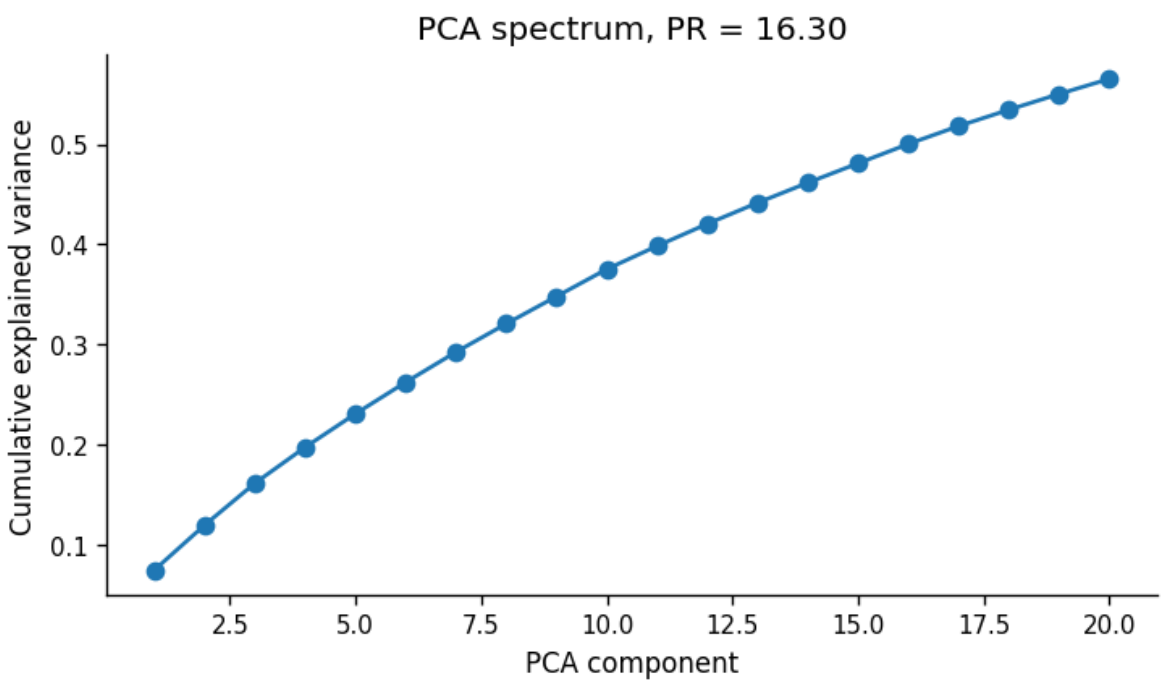
```
c:\Users\Peter\.neuro\Lib\site-packages\umap\umap_.py:1952: UserWarning: n_jo
  warn(
Participation ratio: 16.302
Saved embeddings: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v
```

PCA spectrum

The participation ratio summarizes how many principal directions effectively contribute to the population response. I use it as a simple dimensionality baseline before interpreting nonlinear latent geometry.

```
explained = pd.DataFrame({
    "component": np.arange(1, len(pca.explained_variance_ratio_) + 1),
    "explained_variance_ratio": pca.explained_variance_ratio_,
    "cumulative_explained_variance": np.cumsum(pca.explained_variance_ratio_
})

save_table(
    explained,
```

```
    paths.tables_dir / f"05_pca_explained_variance_session_{session_id}.csv"
)

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(
    explained["component"],
    explained["cumulative_explained_variance"],
    marker="o",
)
ax.set_xlabel("PCA component")
ax.set_ylabel("Cumulative explained variance")
ax.set_title(f"PCA spectrum, PR = {pr:.2f}")

save_figure(
    fig,
    paths.figures_dir / "05_pca_cumulative_explained_variance.png",
)

plt.show()
```



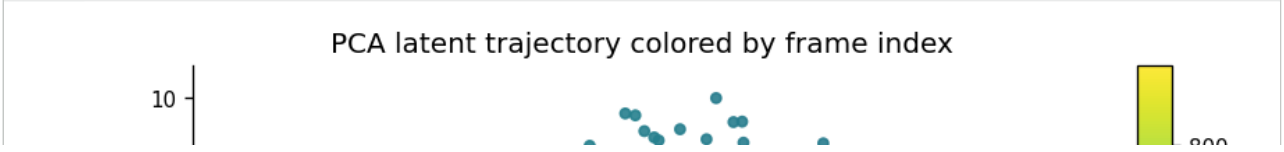
Visualize embeddings with correctly labelled colorbars

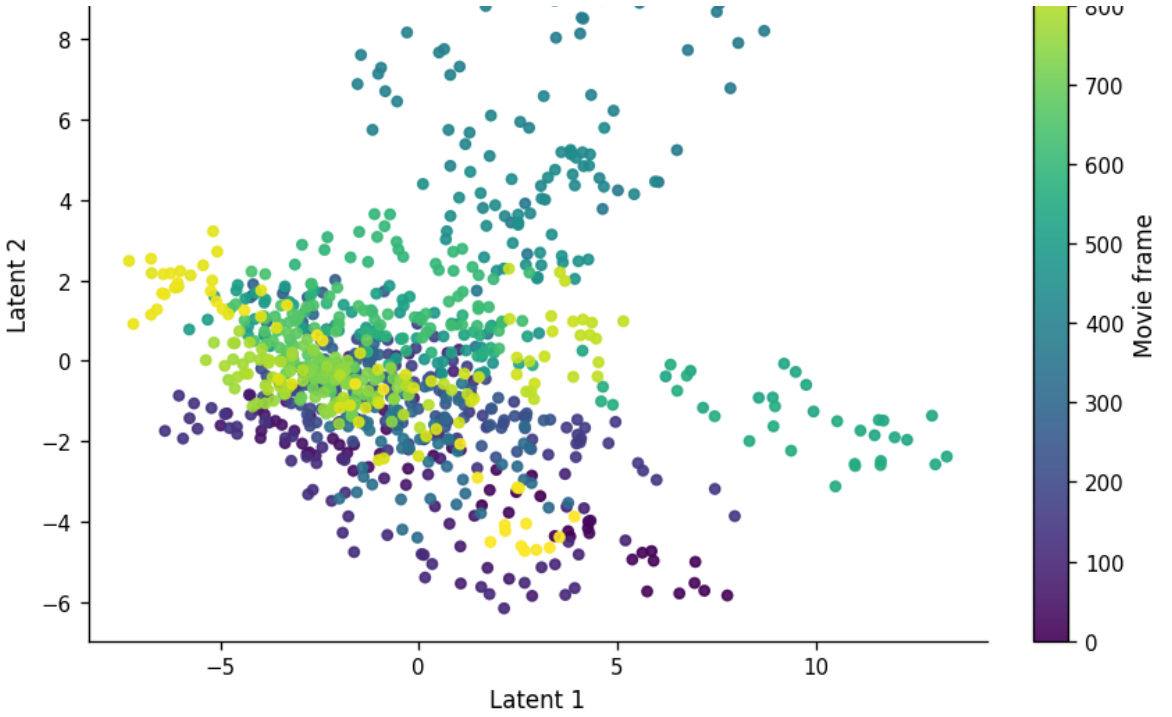
I color each embedding by movie frame index and by real stimulus features. The colorbar label is explicitly tied to the plotted variable so contrast and spatial

colorbar label is explicitly tied to the plotted variable so contrast and spatial-frequency plots are not mislabeled as movie-frame plots.

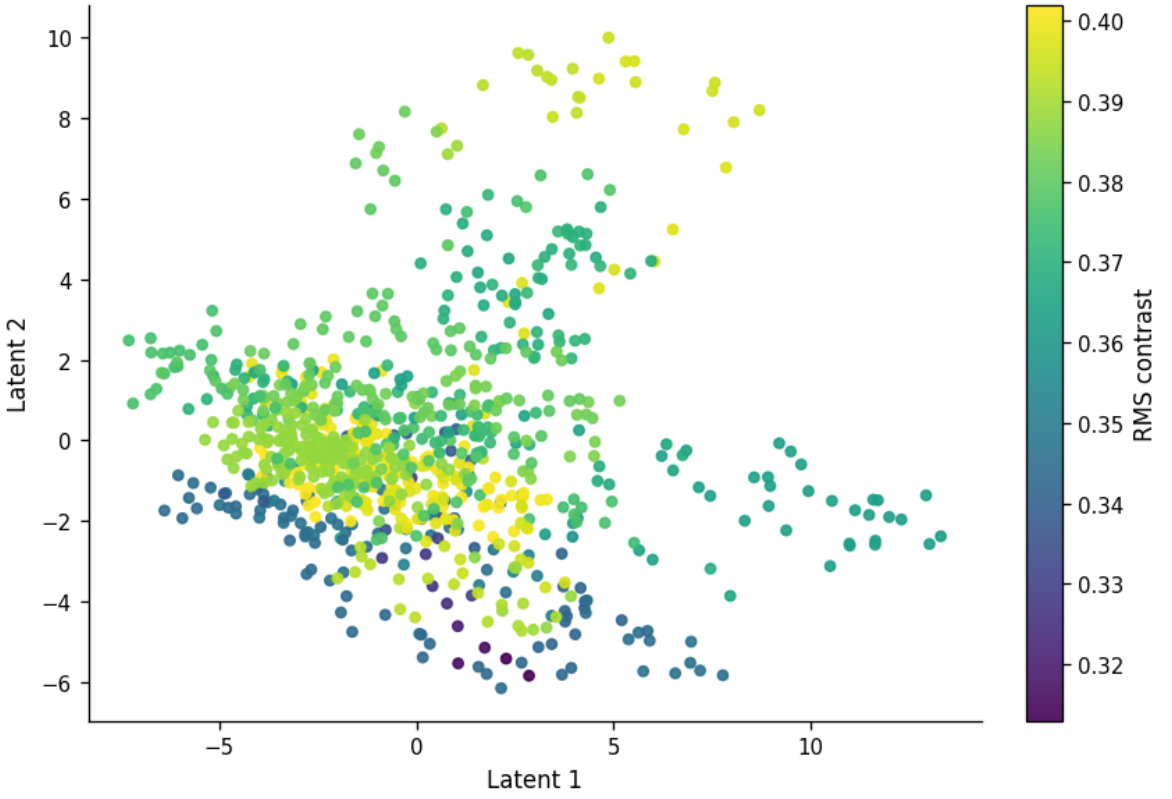
```
    def plot_embedding_colored(
        embedding,
        color_values,
        title,
        colorbar_label,
        save_path,
        point_size=18,
        alpha=0.90,
    ):
        """Plot a 2D embedding with a correctly labelled continuous colorbar."""
        embedding = np.asarray(embedding)
        color_values = np.asarray(color_values)

        if embedding.ndim != 2 or embedding.shape[1] < 2:
            raise ValueError("I need an embedding matrix with at least two laten

        if embedding.shape[0] != color_values.shape[0]:
            raise ValueError(
                f"Embedding has {embedding.shape[0]} rows but color vector has {
            )

        fig, ax = plt.subplots(figsize=(7, 5))

        sc = ax.scatter(
            embedding[:, 0],
            embedding[:, 1],
            c=color_values,
            s=point_size,
            alpha=alpha,
        )

        ax.set_xlabel("Latent 1")
        ax.set_ylabel("Latent 2")
        ax.set_title(title)

        cbar = fig.colorbar(sc, ax=ax)
        cbar.set_label(colorbar_label)

        save_figure(fig, save_path)
        plt.show()

        return fig


    embedding_dict = {
        "pca": pca_embedding,
        "umap": umap_embedding,
```

```
    "isomap": isomap_embedding,
}

color_specs = [
    {
        "column": "movie_frame",
        "values": features["movie_frame"].to_numpy(),
        "label": "Movie frame",
        "slug": "frame_index",
        "title_suffix": "frame index",
    },
    {
        "column": "rms_contrast",
        "values": features["rms_contrast"].to_numpy(),
        "label": "RMS contrast",
        "slug": "rms_contrast",
        "title_suffix": "real movie contrast",
    },
    {
        "column": "spatial_frequency_centroid",
        "values": features["spatial_frequency_centroid"].to_numpy(),
        "label": "Spatial-frequency centroid",
        "slug": "spatial_frequency",
        "title_suffix": "spatial-frequency centroid",
    },
]

if "orientation_selectivity" in features.columns:
    color_specs.append({
        "column": "orientation_selectivity",
        "values": features["orientation_selectivity"].to_numpy(),
        "label": "Orientation selectivity",
        "slug": "orientation_selectivity",
        "title_suffix": "orientation selectivity",
    })

for name, Z in embedding_dict.items():
    for spec in color_specs:
        plot_embedding_colored(
            embedding=Z,
            color_values=spec["values"],
            title=f"{name.upper()} latent trajectory colored by {spec['title
            colorbar_label=spec["label"],
            save_path=paths.figures_dir / f"05_{name}_latent_trajectory_{spe
        )
```

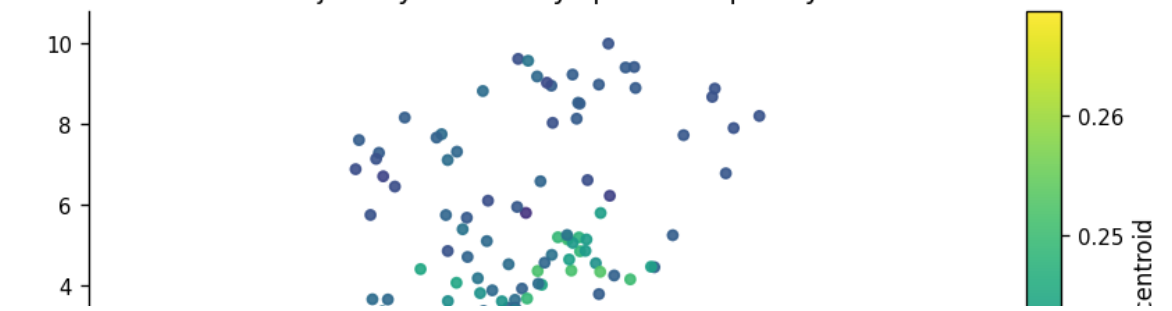


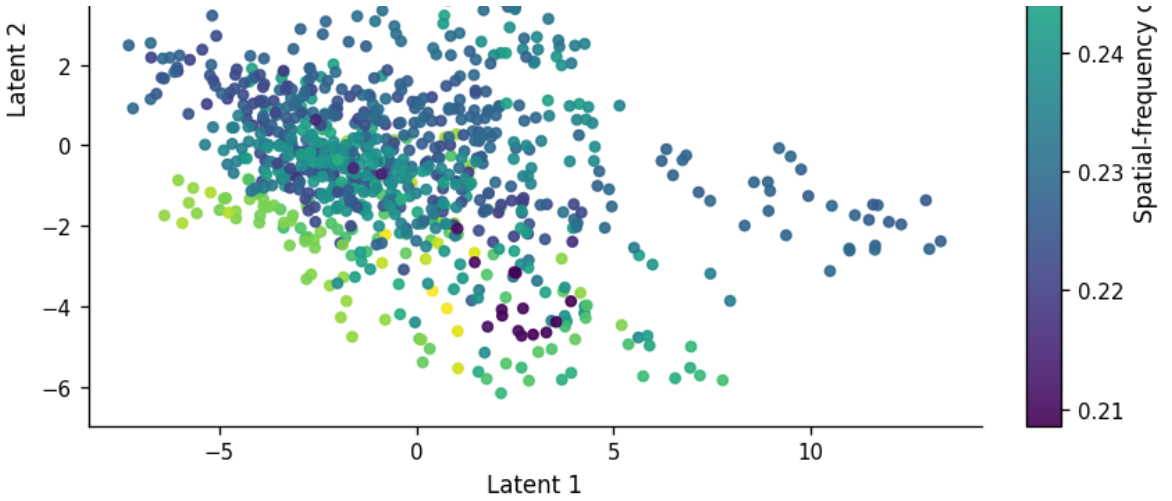


PCA latent trajectory colored by real movie contrast

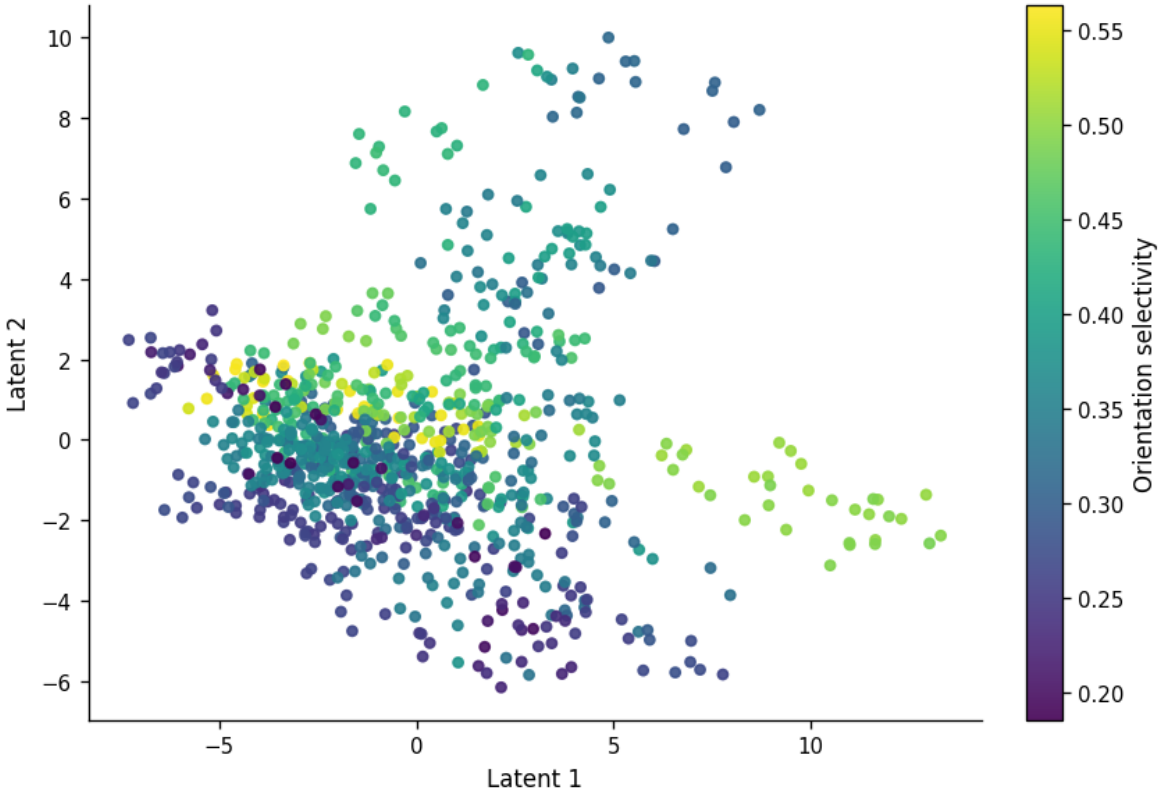


PCA latent trajectory colored by spatial-frequency centroid

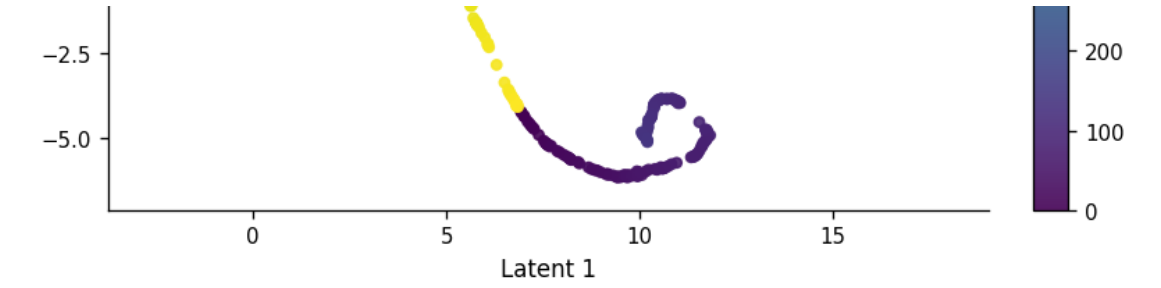


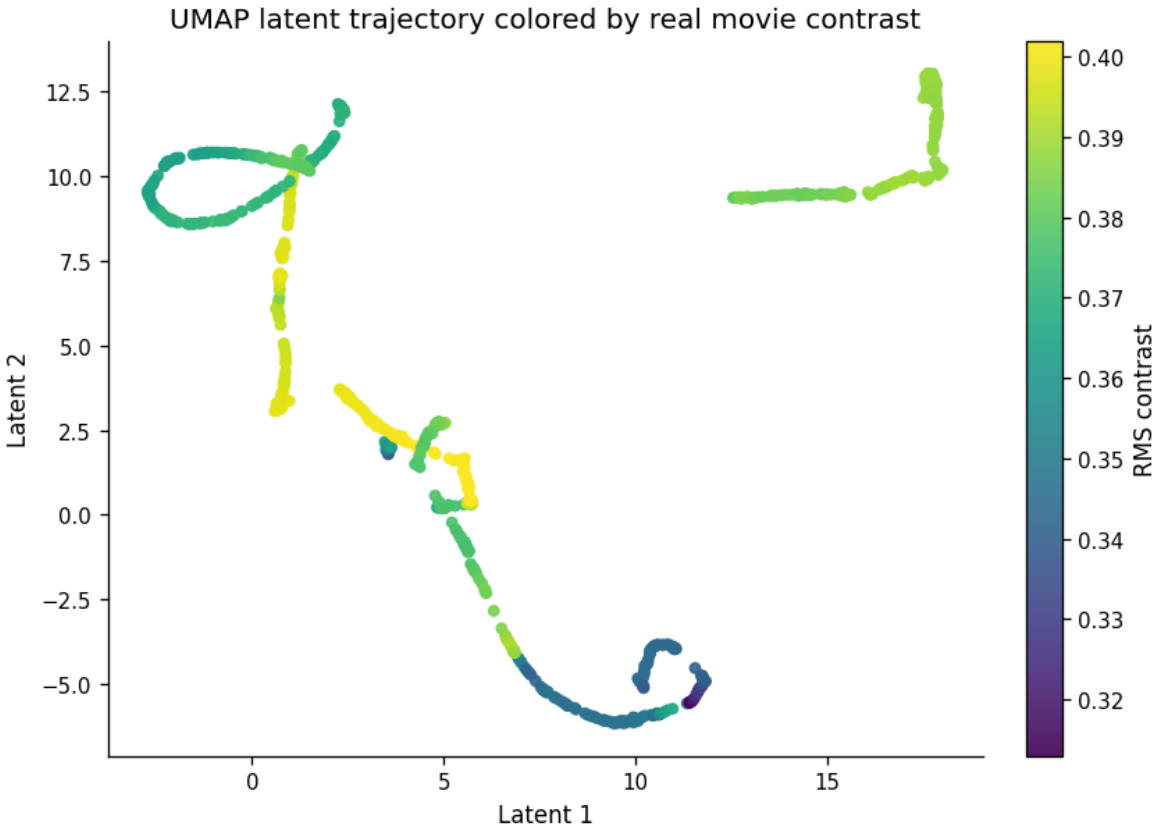


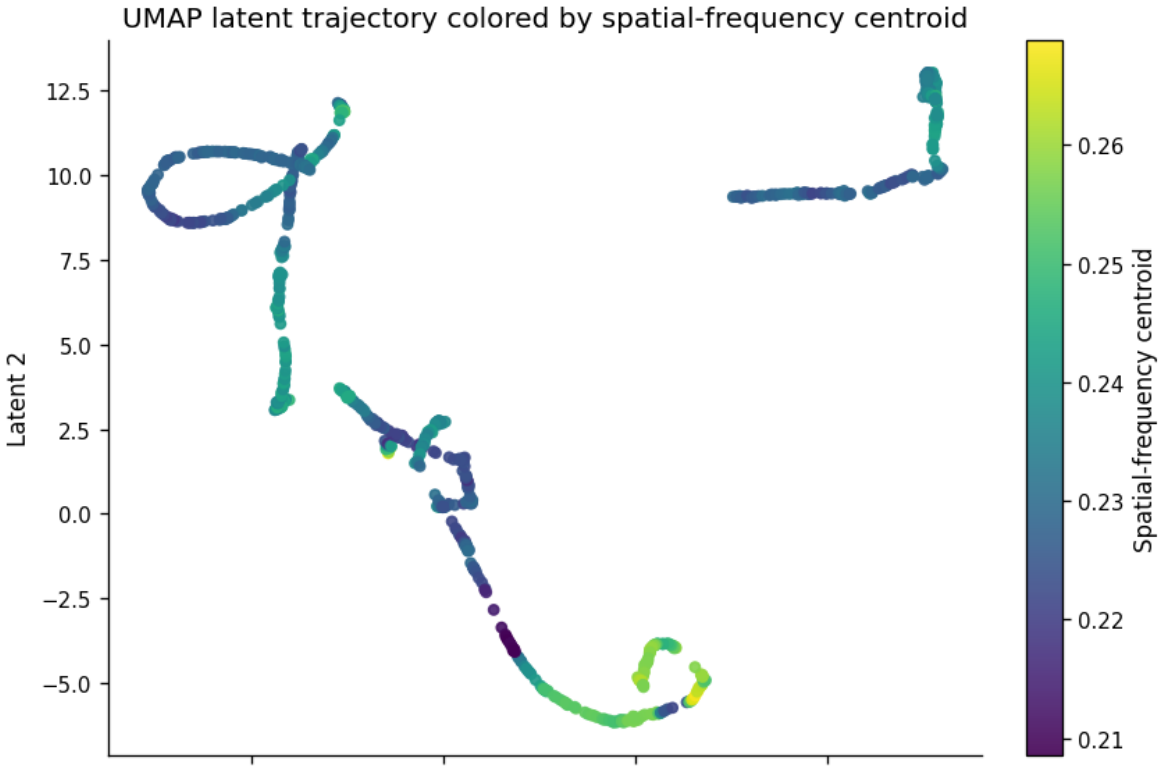
PCA latent trajectory colored by orientation selectivity

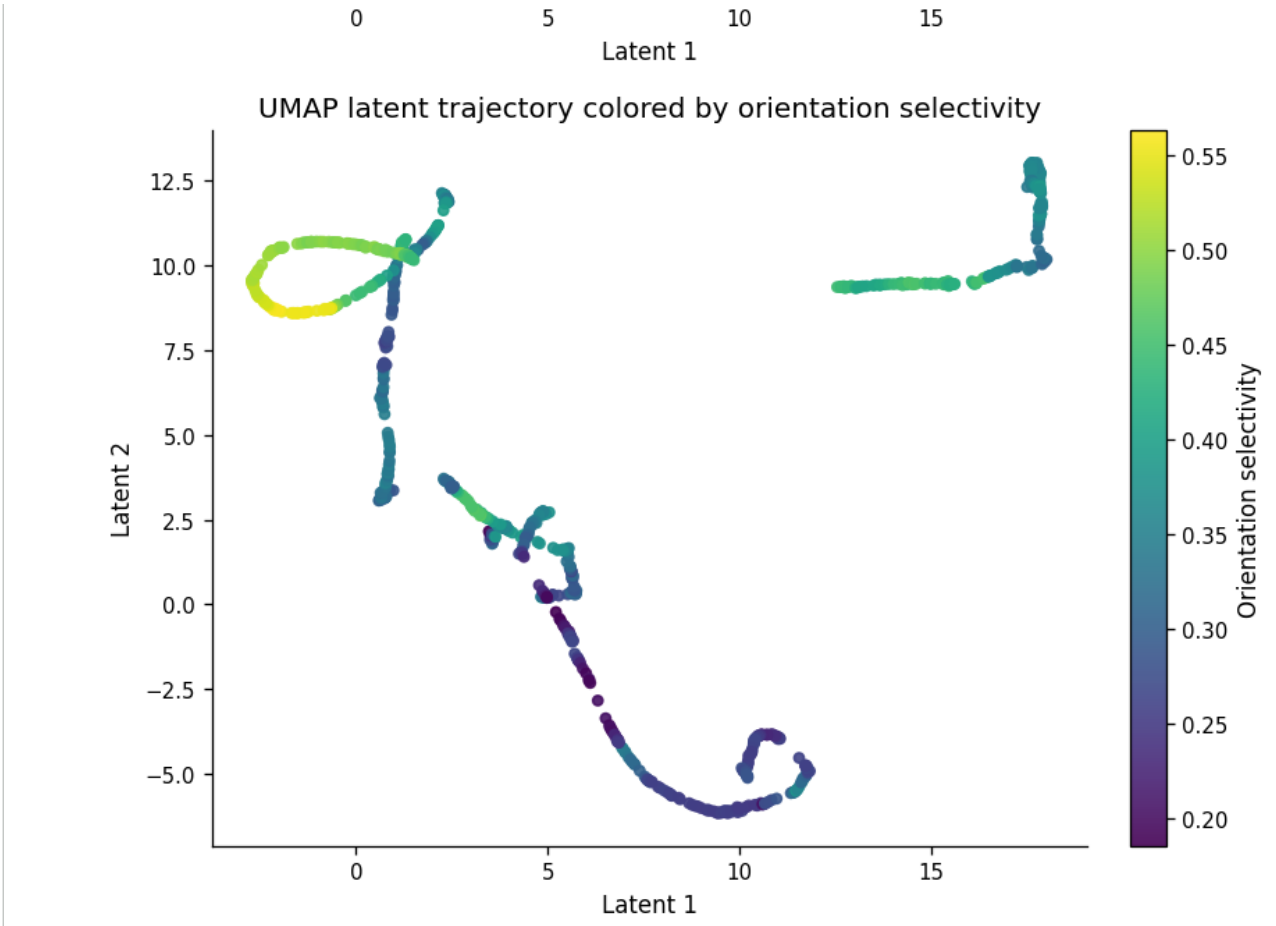


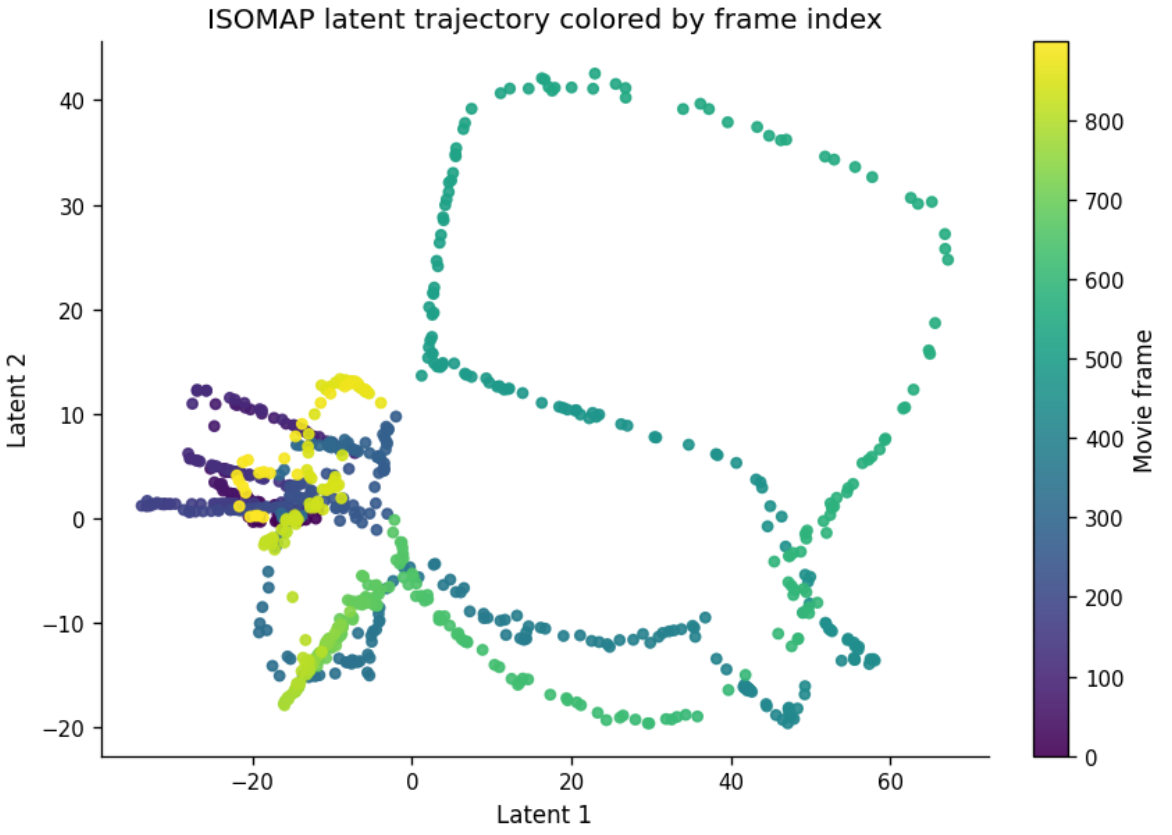
UMAP latent trajectory colored by frame index


−2.5
−5.0
0
5
10
15
Latent 1
200
100
0

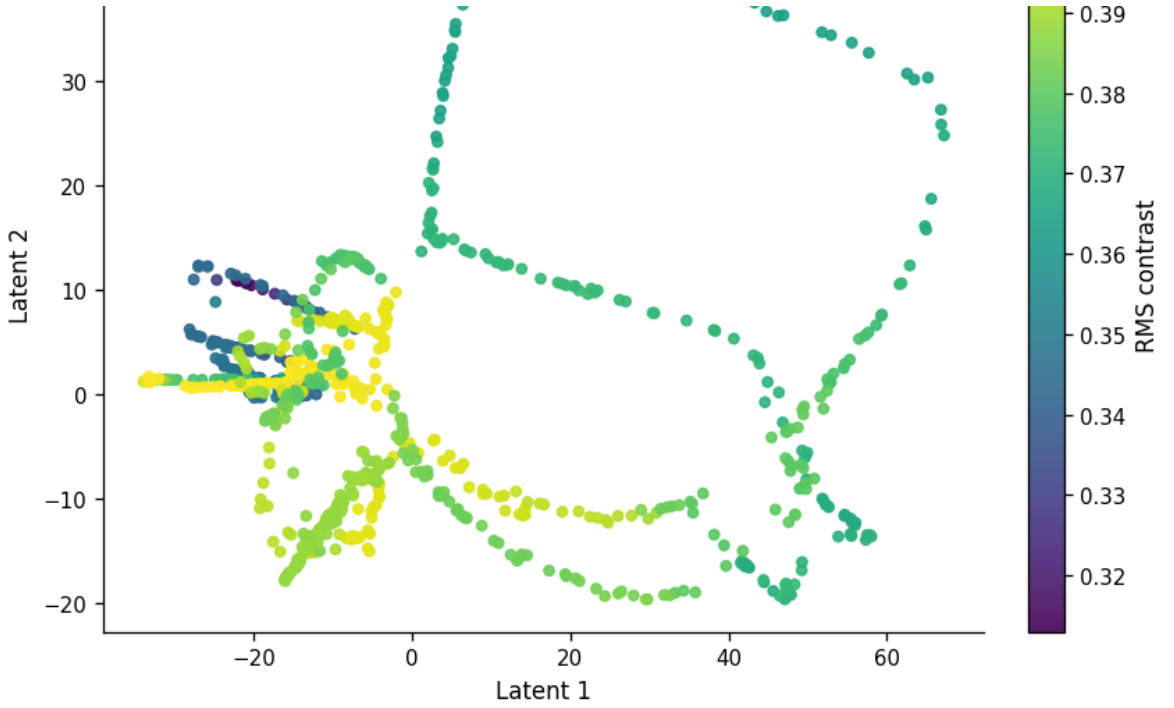

UMAP latent trajectory colored by real movie contrast
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
Latent 2
0
5
10
15
Latent 1
0.40
0.39
0.38
0.37
0.36
0.35
0.34
0.33
0.32
RMS contrast


UMAP latent trajectory colored by spatial-frequency centroid
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
Latent 2
0.26
0.25
0.24
0.23
0.22
0.21
Spatial-frequency centroid

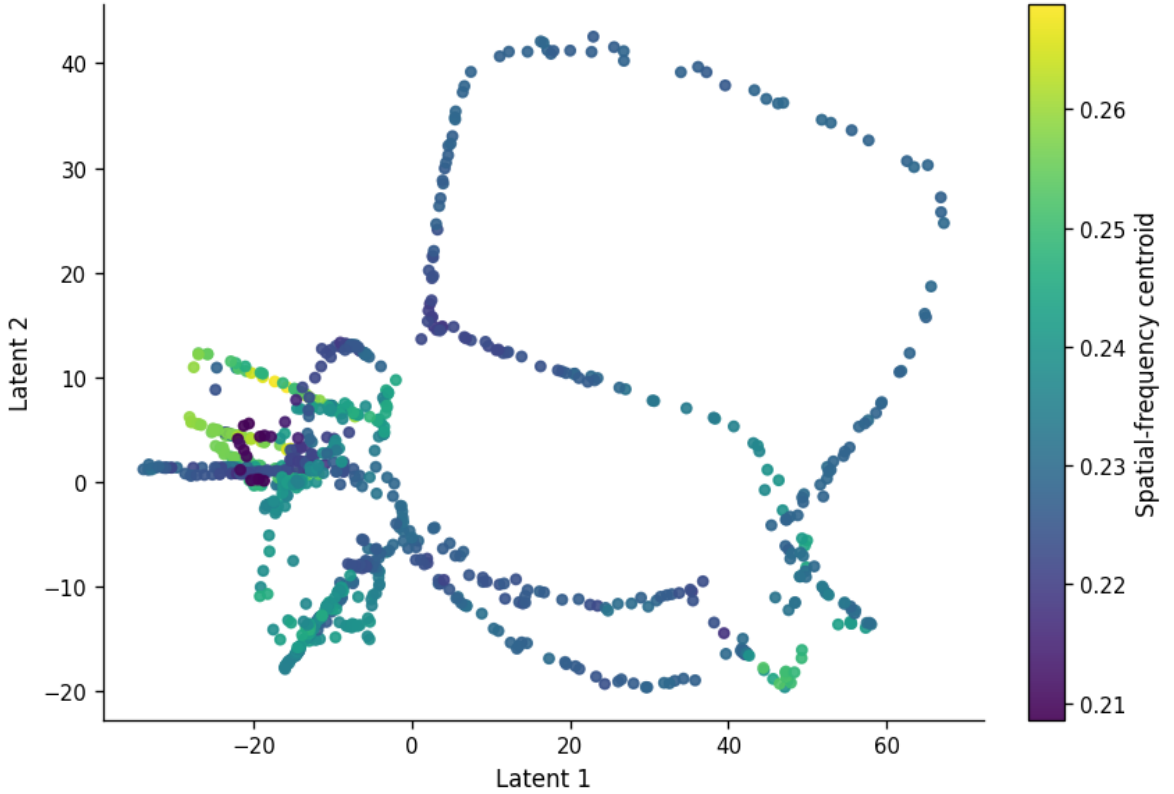

0
5
10
15
Latent 1
UMAP latent trajectory colored by orientation selectivity
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
Latent 2
0
5
10
15
Latent 1
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.20
Orientation selectivity


ISOMAP latent trajectory colored by frame index
40
30
20
10
0
−10
−20
Latent 2
−20
0
20
40
60
Latent 1
800
700
600
500
400
300
200
100
0
Movie frame

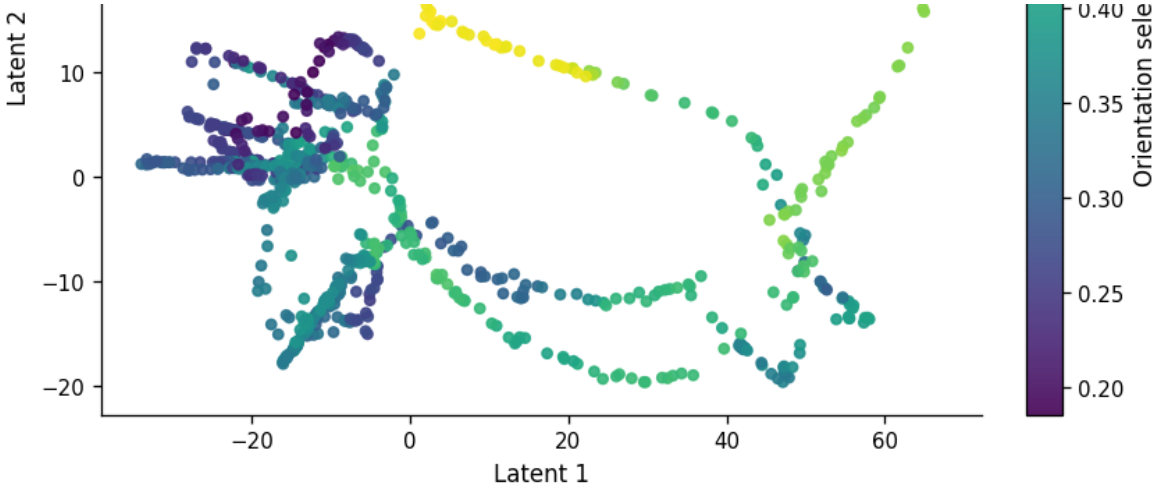

ISOMAP latent trajectory colored by real movie contrast
40
0.40



ISOMAP latent trajectory colored by spatial-frequency centroid



ISOMAP latent trajectory colored by orientation selectivity


Latent 2
10
0
−10
−20
−20
0
20
40
60
Latent 1
0.40
0.35
0.30
0.25
0.20
Orientation sele

Quantify whether latent coordinates predict real movie features

The scatter plots are useful but not enough. I quantify whether each embedding predicts real continuous movie-frame features using block-wise cross-validation over contiguous movie segments.

This is stricter than random frame-wise cross-validation because adjacent natural movie frames are highly autocorrelated.

```
from v1_manifold.evaluation import (
    make_contiguous_frame_blocks,
    frame_time_features,
    evaluate_feature_regression,
)
```

```
continuous_targets = [
    "rms_contrast",
    "spatial_frequency_centroid",
    "orientation_selectivity",
    "luminance_std",
    "total_spectral_power",
]

continuous_targets = [target for target in continuous_targets if target in f

frame_blocks = make_contiguous_frame_blocks(
    n_samples=len(features),
    n_blocks=int(cfg.get("evaluation", {}).get("n_movie_blocks", 5)),
)

latent_representations = {
    "pca_3d": pca_embedding,
    "pca_20d": pca_scores[:, :min(20, pca_scores.shape[1])],
    "umap_3d": umap_embedding,
    "isomap_3d": isomap_embedding,
}

latent_feature_scores = evaluate_feature_regression(
    latent_representations,
    features,
    continuous_targets,
    groups=frame_blocks,
    alpha=float(cfg.get("evaluation", {}).get("latent_regression_alpha", 1.0
)

save_table(
    latent_feature_scores,
    paths.tables_dir / "05_latent_embedding_real_feature_regression.csv",
)

display(
    latent_feature_scores.sort_values(
        ["target", "mean_r2"],
        ascending=[True, False],
    )
)
```

	representation	target	cv_strategy	n_
13	umap_3d	luminance_std	contiguous_movie_block_groupkfold	
8	pca_20d	luminance_std	contiguous_movie_block_groupkfold	
3	pca_3d	luminance_std	contiguous_movie_block_groupkfold	

18	isomap_3d	luminance_std	contiguous_movie_block_groupkfold
12	umap_3d	orientation_selectivity	contiguous_movie_block_groupkfold
17	isomap_3d	orientation_selectivity	contiguous_movie_block_groupkfold
2	pca_3d	orientation_selectivity	contiguous_movie_block_groupkfold
7	pca_20d	orientation_selectivity	contiguous_movie_block_groupkfold
10	umap_3d	rms_contrast	contiguous_movie_block_groupkfold
5	pca_20d	rms_contrast	contiguous_movie_block_groupkfold
0	pca_3d	rms_contrast	contiguous_movie_block_groupkfold
15	isomap_3d	rms_contrast	contiguous_movie_block_groupkfold
16	isomap_3d	spatial_frequency_centroid	contiguous_movie_block_groupkfold
1	pca_3d	spatial_frequency_centroid	contiguous_movie_block_groupkfold
11	umap_3d	spatial_frequency_centroid	contiguous_movie_block_groupkfold
6	pca_20d	spatial_frequency_centroid	contiguous_movie_block_groupkfold
4	pca_3d	total_spectral_power	contiguous_movie_block_groupkfold
19	isomap_3d	total_spectral_power	contiguous_movie_block_groupkfold
9	pca_20d	total_spectral_power	contiguous_movie_block_groupkfold
14	umap_3d	total_spectral_power	contiguous_movie_block_groupkfold

Compare embeddings with temporal and scalar neural baselines

A structured latent trajectory can arise because the natural movie itself is temporally continuous. I therefore compare PCA, UMAP, and ISOMAP against frame-index baselines and a scalar population-energy baseline.

This does not prove causality, but it prevents me from overinterpreting a visually smooth manifold as stimulus-feature coding without a quantitative control.

```
baseline_representations = {
    "frame_index_linear_baseline": features["movie_frame"].to_numpy().reshap
    "frame_index_fourier_baseline": frame_time_features(
        features["movie_frame"].to_numpy(),
        n_harmonics=int(cfg.get("evaluation", {}).get("temporal_fourier_harm
    ),
    "population_l2_norm_baseline": features["population_l2_norm"].to_numpy()
```

```
}

baseline_scores = evaluate_feature_regression(
    baseline_representations,
    features,
    continuous_targets,
    groups=frame_blocks,
    alpha=float(cfg.get("evaluation", {}).get("latent_regression_alpha", 1.0
)

save_table(
    baseline_scores,
    paths.tables_dir / "05_frame_index_and_population_energy_baselines.csv",
)

latent_vs_baseline = pd.concat(
    [latent_feature_scores, baseline_scores],
    ignore_index=True,
)

save_table(
    latent_vs_baseline,
    paths.tables_dir / "05_latent_vs_time_feature_prediction.csv",
)

display(
    latent_vs_baseline.sort_values(
        ["target", "mean_r2"],
        ascending=[True, False],
    )
)
```

	representation	target	cv_st
13	umap_3d	luminance_std	contiguous_movie_block_gr
33	population_l2_norm_baseline	luminance_std	contiguous_movie_block_gr
8	pca_20d	luminance_std	contiguous_movie_block_gr
3	pca_3d	luminance_std	contiguous_movie_block_gr
18	isomap_3d	luminance_std	contiguous_movie_block_gr
23	frame_index_linear_baseline	luminance_std	contiguous_movie_block_gr
28	frame_index_fourier_baseline	luminance_std	contiguous_movie_block_gr
12	umap_3d	orientation_selectivity	contiguous_movie_block_gr
17	isomap_3d	orientation_selectivity	contiguous_movie_block_gr
2	pca_3d	orientation_selectivity	contiguous_movie_block_gr

32	population_l2_norm_baseline	orientation_selectivity	contiguous_movie_block_gr
7	pca_20d	orientation_selectivity	contiguous_movie_block_gr
22	frame_index_linear_baseline	orientation_selectivity	contiguous_movie_block_gr
27	frame_index_fourier_baseline	orientation_selectivity	contiguous_movie_block_gr
10	umap_3d	rms_contrast	contiguous_movie_block_gr
30	population_l2_norm_baseline	rms_contrast	contiguous_movie_block_gr
5	pca_20d	rms_contrast	contiguous_movie_block_gr
0	pca_3d	rms_contrast	contiguous_movie_block_gr
15	isomap_3d	rms_contrast	contiguous_movie_block_gr
20	frame_index_linear_baseline	rms_contrast	contiguous_movie_block_gr
25	frame_index_fourier_baseline	rms_contrast	contiguous_movie_block_gr
21	frame_index_linear_baseline	spatial_frequency_centroid	contiguous_movie_block_gr
16	isomap_3d	spatial_frequency_centroid	contiguous_movie_block_gr
31	population_l2_norm_baseline	spatial_frequency_centroid	contiguous_movie_block_gr
1	pca_3d	spatial_frequency_centroid	contiguous_movie_block_gr
11	umap_3d	spatial_frequency_centroid	contiguous_movie_block_gr
6	pca_20d	spatial_frequency_centroid	contiguous_movie_block_gr
26	frame_index_fourier_baseline	spatial_frequency_centroid	contiguous_movie_block_gr
4	pca_3d	total_spectral_power	contiguous_movie_block_gr
34	population_l2_norm_baseline	total_spectral_power	contiguous_movie_block_gr
19	isomap_3d	total_spectral_power	contiguous_movie_block_gr
9	pca_20d	total_spectral_power	contiguous_movie_block_gr
14	umap_3d	total_spectral_power	contiguous_movie_block_gr
24	frame_index_linear_baseline	total_spectral_power	contiguous_movie_block_gr
29	frame_index_fourier_baseline	total_spectral_power	contiguous_movie_block_gr

Summarize whether neural embeddings add information beyond time

For each real movie feature, I compare the best latent embedding against the best frame-index baseline. A positive delta means the neural embedding predicts that

frame-index baseline. A positive delta means the neural embedding predicts that feature better than the temporal baseline under the same block-wise split.

```
is_temporal_baseline = latent_vs_baseline["representation"].isin([
    "frame_index_linear_baseline",
    "frame_index_fourier_baseline",
])

best_temporal = (
    latent_vs_baseline[is_temporal_baseline]
    .sort_values(["target", "mean_r2"], ascending=[True, False])
    .groupby("target", as_index=False)
    .first()
    .rename(columns={
        "representation": "best_temporal_baseline",
        "mean_r2": "best_temporal_mean_r2",
        "mean_mae": "best_temporal_mean_mae",
        "mean_spearman_r": "best_temporal_mean_spearman_r",
    })
)

best_latent = (
    latent_vs_baseline[~is_temporal_baseline]
    .query("representation != 'population_l2_norm_baseline'")
    .sort_values(["target", "mean_r2"], ascending=[True, False])
    .groupby("target", as_index=False)
    .first()
    .rename(columns={
        "representation": "best_latent_representation",
        "mean_r2": "best_latent_mean_r2",
        "mean_mae": "best_latent_mean_mae",
        "mean_spearman_r": "best_latent_mean_spearman_r",
    })
)

latent_gain = best_latent.merge(
    best_temporal,
    on="target",
    how="left",
)

latent_gain["delta_r2_vs_best_temporal"] = (
    latent_gain["best_latent_mean_r2"] - latent_gain["best_temporal_mean_r2"
)

latent_gain["delta_spearman_vs_best_temporal"] = (
    latent_gain["best_latent_mean_spearman_r"] - latent_gain["best_temporal_
)

save_table(
```

```
    latent_gain,
    paths.tables_dir / "05_latent_gain_over_temporal_baseline.csv",
)

display(latent_gain)
```

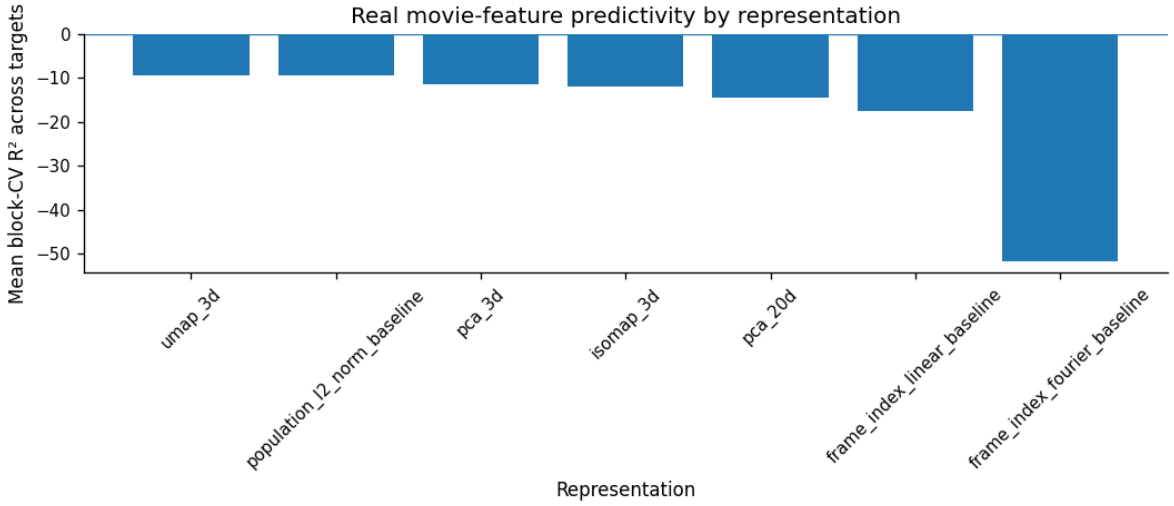
	target	best_latent_representation	cv_st
0	luminance_std	umap_3d	contiguous_movie_block_g
1	orientation_selectivity	umap_3d	contiguous_movie_block_g
2	rms_contrast	umap_3d	contiguous_movie_block_g
3	spatial_frequency_centroid	isomap_3d	contiguous_movie_block_g
4	total_spectral_power	pca_3d	contiguous_movie_block_g

5 rows × 27 columns

```
plot_df = latent_vs_baseline.copy()

# I aggregate over targets to obtain a compact method-level summary.
method_summary = (
    plot_df
    .groupby("representation", as_index=False)
    .agg(
        mean_r2_across_targets=("mean_r2", "mean"),
        mean_spearman_across_targets=("mean_spearman_r", "mean"),
        n_targets=("target", "nunique"),
    )
    .sort_values("mean_r2_across_targets", ascending=False)
)

save_table(
    method_summary,
    paths.tables_dir / "05_representation_method_summary.csv",
)

fig, ax = plt.subplots(figsize=(9, 4))

ax.bar(
    method_summary["representation"],
    method_summary["mean_r2_across_targets"],
)

ax.axhline(0, linewidth=0.8)
ax.set_xlabel("Representation")
ax.set_ylabel("Mean block-CV R² across targets")
```

```
ax.set_title("Real movie-feature predictivity by representation")
ax.tick_params(axis="x", rotation=45)

fig.tight_layout()

save_figure(
    fig,
    paths.figures_dir / "05_representation_feature_predictivity_summary.png"
)

plt.show()

display(method_summary)
```

```
C:\Users\Peter\AppData\Local\Temp\ipykernel_40732\588070037.py:33: UserWarnin
  fig.tight_layout()
```



	representation	mean_r2_across_targets	mean_spearman_across_tar
6	umap_3d	-9.277270	0.09
5	population_l2_norm_baseline	-9.431165	0.00
4	pca_3d	-11.514799	0.03
2	isomap_3d	-11.837900	0.00
3	pca_20d	-14.466426	0.10
1	frame_index_linear_baseline	-17.417848	-0.15
0	frame_index_fourier_baseline	-51.760036	0.12